LAPRAK TEKNOLOGI IOT II

PRATIKUM 1

Oleh:

NAMA : TAUFIK FADILLA

NPM : 25782114

KELAS : TRI – 3D

Dosen:

Dr. Ir. Septafiansyah Dwi Putra, S.T., M.T.

Agus Ambarwari, S.Pd., M.Kom.

Andi Saptono, S.Kom

TEKNOLOGI REKAYASA INTERNET

JURUSAN TEKNOLOGI INFORMASI

POLITEKNIK NEGERI LAMPUNG

2025/2026

1.5 Praktikum

1.5.1 Praktikum 1: *Digital Output* (Menyalakan LED)

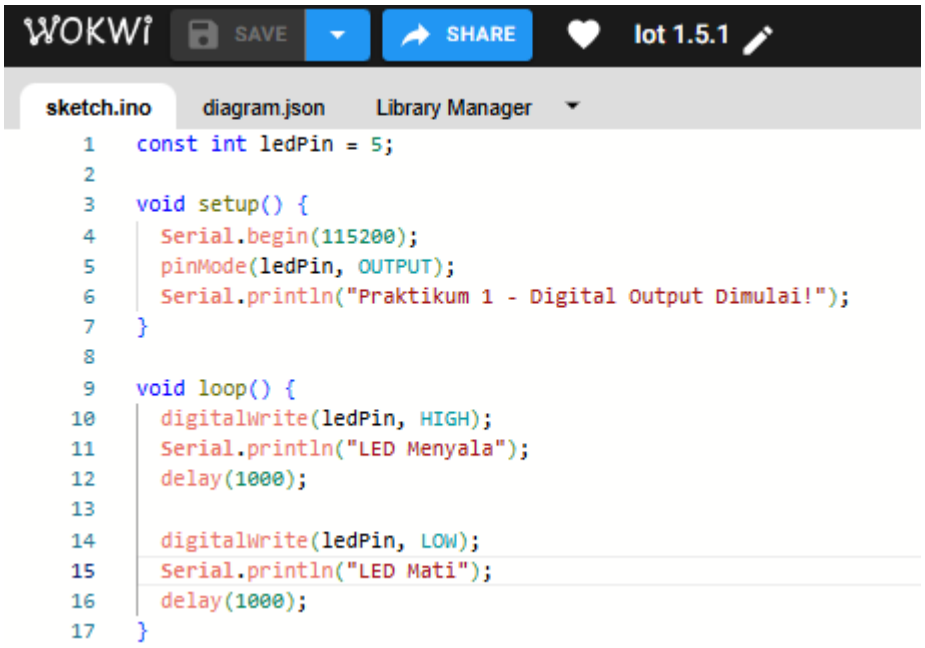
TUJUAN

Memprogram board NodeMCU agar dapat mengalirkan tegangan 3.3V (HIGH) secara terprogram untuk menyalakan komponen eksternal (LED).

ALAT DAN BAHAN

1. Board NodeMCU (ESP8266 / ESP32)
2. Breadboard
3. Komponen LED
4. Resistor 200 Ohm
5. Kabel Jumper
6. Komputer/Laptop (beserta Arduino IDE atau Wokwi Simulator)

KODE PROGRAM




```
1   const int ledPin = 5;
2
3   void setup() {
4     Serial.begin(115200);
5     pinMode(ledPin, OUTPUT);
6     Serial.println("Praktikum 1 - Digital Output Dimulai!");
7   }
8
9   void loop() {
10    digitalWrite(ledPin, HIGH);
11    Serial.println("LED Menyala");
12    delay(1000);
13
14    digitalWrite(ledPin, LOW);
15    Serial.println("LED Mati");
16    delay(1000);
17  }
```

CARA KERJA PROGRAM

1. **Inisialisasi Rangkaian:** NodeMCU dihubungkan dengan rangkaian LED, di mana pin D1 (GPIO 5) tersambung ke anoda LED, sedangkan katoda terhubung ke ground (GND) melalui resistor 220 Ohm yang berfungsi sebagai pembatas arus.
2. **Setup Awal:** Saat program pertama kali dijalankan, NodeMCU akan mengeksekusi blok setup() untuk memulai komunikasi dengan Serial Monitor pada *baud rate* 115200 dan mengatur pin GPIO 5 sebagai jalur *output*.
3. **Siklus Menyala (HIGH):** Program masuk ke dalam blok loop(). NodeMCU mengirimkan tegangan 3.3V ke pin GPIO 5, yang memicu arus mengalir sehingga LED menyala.

Bersamaan dengan itu, pesan "LED Menyala" dikirimkan ke Serial Monitor. Kondisi ini dipertahankan selama 1 detik (1000 ms).

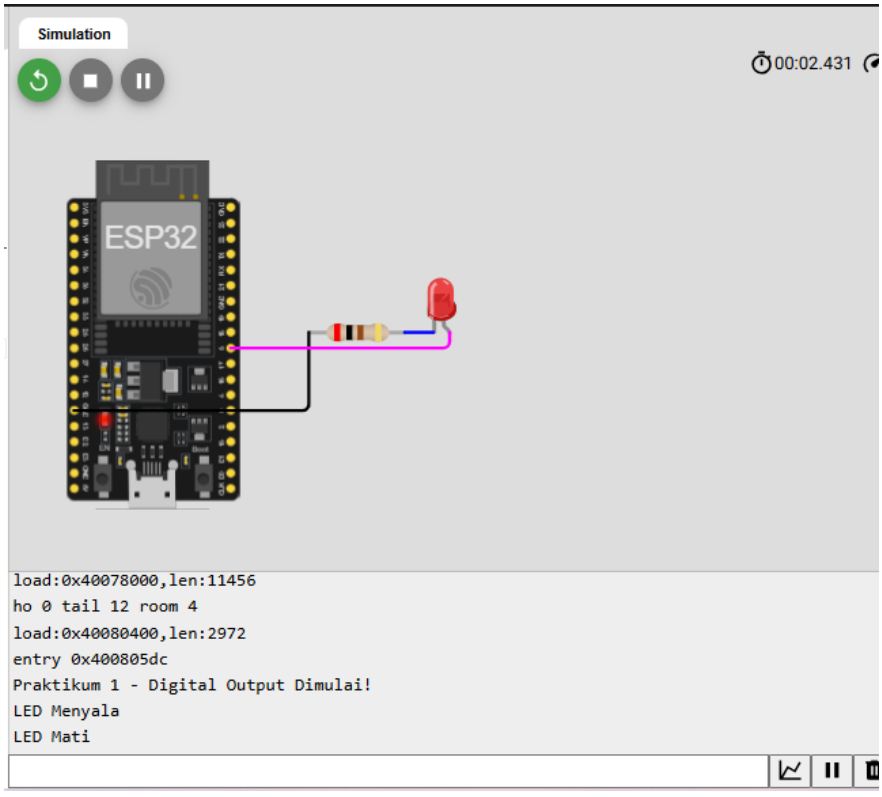
4. **Siklus Mati (LOW):** Setelah 1 detik, NodeMCU memutus tegangan pada pin GPIO 5 menjadi 0V, sehingga LED mati. Pesan "LED Mati" dikirimkan ke Serial Monitor. Kondisi ini juga dipertahankan selama 1 detik.
5. **Perulangan:** Karena perintah berada di dalam fungsi loop(), langkah 3 dan 4 akan terus diulang selama mikrokontroler terhubung dengan daya.

PENJELASAN SINGKAT KODE

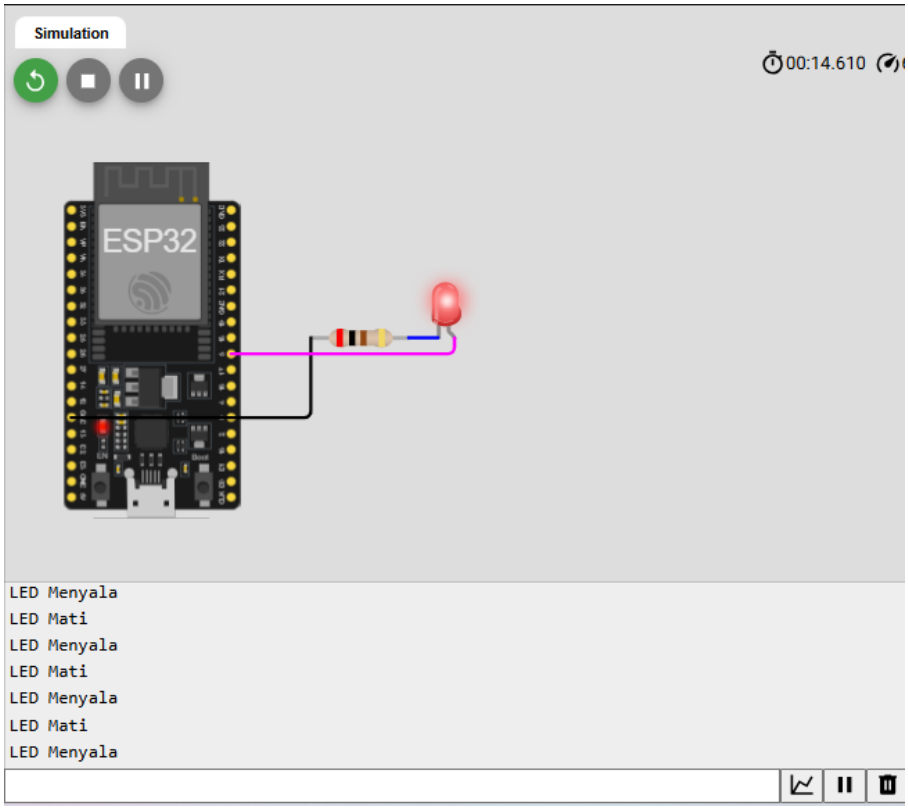
- setup(): Blok fungsi yang dieksekusi hanya satu kali saat NodeMCU pertama kali dinyalakan atau di-*reset*. Umumnya digunakan untuk pengaturan awal (*initialization*).
- loop(): Blok fungsi yang akan dieksekusi secara berulang-ulang (terus-menerus) selama NodeMCU memiliki sumber daya.
- pinMode(ledPin, OUTPUT): Memberi instruksi bahwa pin yang digunakan (dalam hal ini GPIO 5) akan difungsikan sebagai jalur keluar (*output*) tegangan.
- digitalWrite(ledPin, HIGH): Memberikan sinyal digital berupa tegangan 3.3V ke pin yang telah ditentukan, sehingga arus listrik dapat mengalir dan menyalakan LED.
- delay(1000): Menghentikan eksekusi baris program selanjutnya (memberikan jeda) selama 1000 milidetik (1 detik).

GAMBAR

Saat led mati.



Saat led hidup.



HASIL

LED berkedip secara periodik dengan jeda 1 detik (1 detik menyala, 1 detik mati). Saat dipantau melalui Serial Monitor dengan pengaturan *baud rate* 115200, akan muncul teks indikator "LED Menyala" dan "LED Mati" yang dicetak selaras dengan kondisi fisik (atau simulasi) LED yang sedang berkedip.

1.5.2 Praktikum 2: *Digital Input* (Membaca *Push Button*)

TUJUAN

Mendeteksi interaksi fisik dengan membaca status tegangan dari sebuah tombol, memanfaatkan konfigurasi rangkaian resistor *pull-down* eksternal.

ALAT DAN BAHAN

1. Board NodeMCU (ESP8266 / ESP32)
2. Breadboard
3. *Push Button* (Tombol Tekan)
4. Komponen LED
5. Resistor 10k Ohm (sebagai *pull-down* untuk tombol)
6. Resistor 200 Ohm (sebagai pembatas arus untuk LED)
7. Kabel Jumper
8. Komputer/Laptop (beserta Arduino IDE atau Wokwi Simulator)

KODE PROGRAM

WOKWi SAVE SHARE Iot 1.5.2

sketch.ino diagram.json Library Manager

```
const int buttonPin = 25;
const int ledPin = 26;

int buttonState = 0;

void setup() {
  Serial.begin(115200);
  pinMode(buttonPin, INPUT);
  pinMode(ledPin, OUTPUT);
}

void loop() {
  buttonState = digitalRead(buttonPin);

  if (buttonState == HIGH) {
    digitalWrite(ledPin, HIGH);
    Serial.println("Tombol ditekan! -> LED ON");
  } else {
    digitalWrite(ledPin, LOW);
  }
}
```

CARA KERJA PROGRAM

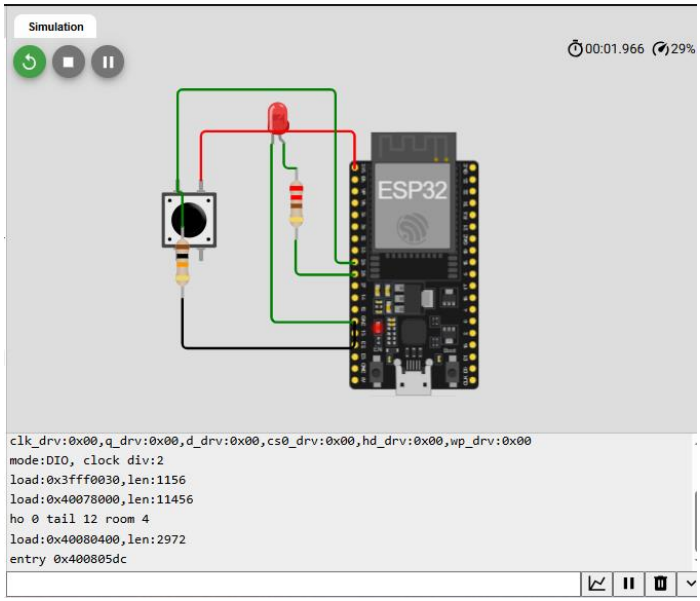
1. **Inisialisasi Rangkaian:** Rangkaian membaca input dari *push button* yang terhubung ke pin D2 (GPIO 4) dan 3.3V. Kaki tombol yang menuju GPIO 4 juga dihubungkan ke GND melalui resistor 10k Ohm (*pull-down*). Rangkaian *output* tetap menggunakan LED pada pin D1 (GPIO 5) dari praktikum sebelumnya.
2. **Setup Awal:** Pada blok setup(), mikrokontroler memulai komunikasi serial dan menetapkan mode pin: GPIO 4 sebagai INPUT untuk membaca status tombol, dan GPIO 5 sebagai OUTPUT untuk mengendalikan LED.
3. **Pembacaan Status Tombol:** Di dalam blok loop(), mikrokontroler terus-menerus mengecek status tegangan pada pin GPIO 4 menggunakan fungsi digitalRead(). Variabel buttonState akan menyimpan hasil pembacaan tersebut (HIGH atau LOW).
4. **Logika Kondisional (Tombol Ditekan):** Saat tombol fisik ditekan, arus 3.3V mengalir langsung ke pin GPIO 4. Mikrokontroler mendeteksi adanya tegangan (HIGH). Kondisi if (buttonState == HIGH) terpenuhi, sehingga mikrokontroler memberikan perintah menyalakan LED dan mengirimkan pesan "Tombol ditekan! -> LED ON" ke Serial Monitor.
5. **Logika Kondisional (Tombol Dilepas):** Saat tombol dilepaskan, jalur 3.3V terputus dan pin GPIO 4 langsung terhubung ke *Ground* (GND) melalui resistor *pull-down* 10k Ohm. Mikrokontroler membaca status tegangan menjadi LOW. Kondisi masuk ke blok else, dan mikrokontroler memberikan perintah untuk mematikan LED.

PENJELASAN SINGKAT KODE

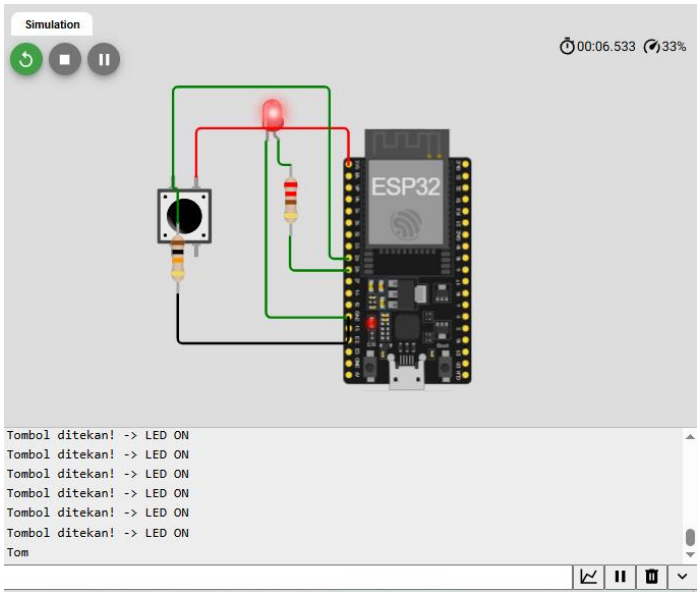
- pinMode(buttonPin, INPUT): Menetapkan pin D2 (GPIO 4) sebagai jalur masuk (*Input*) yang bertugas mendeteksi atau membaca sinyal tegangan dari luar (dari *push button*).
- digitalRead(buttonPin): Fungsi yang bertugas membaca status tegangan pada pin saat ini. Hasil kembaliannya akan berupa HIGH (jika ada tegangan masuk) atau LOW (jika tidak ada tegangan).
- if (buttonState == HIGH): Blok logika kondisional utama. Karena menggunakan rangkaian resistor *pull-down*, penekanan tombol akan mengalirkan tegangan 3.3V ke pin mikrokontroler. Akibatnya, status terbaca HIGH dan blok instruksi di dalamnya (menyalakan LED dan mencetak teks) akan dijalankan.

GAMBAR

Saat pertama kali di play dan tidak menekan push button.



Saat push button di tekan dan ditahan.



HASIL

Sistem berhasil mendeteksi interaksi fisik secara *real-time*. Saat tombol pada rangkaian ditekan dan ditahan, LED akan menyala terang dan Serial Monitor menampilkan tulisan "Tombol ditekan! -> LED ON". Saat tombol dilepaskan, LED langsung padam tanpa *delay*, sesuai dengan logika else pada program.

1.6 Latihan

- **Latihan 1 (Pemahaman Logika Pin):** Berdasarkan tabel Referensi Pinout di subbab 1.3.3, jelaskan apa yang mungkin terjadi jika Anda menggunakan logika rangkaian dari Praktikum 2 (Active-High / *pull-down*), namun menyambungkan tombol tersebut ke pin **D8 (GPIO 15)**, lalu menekan tombol tersebut *tepat* pada detik di mana NodeMCU baru diberi daya?

  Label pin D8 terhubung ke GPIO 15.

  Pin D8 (GPIO 15) memiliki karakteristik perilaku di mana proses *boot* akan gagal jika pin tersebut ditarik ke logika HIGH.

  Pada rangkaian logika *Active-High* menggunakan *pull-down*, menekan tombol akan mengalirkan arus 3.3V langsung ke pin mikrokontroler sehingga statusnya terbaca HIGH (1).

  Oleh karena itu, jika tombol ditekan tepat pada detik mikrokontroler diberi daya, pin D8 (GPIO 15) akan menerima tegangan HIGH dan menyebabkan NodeMCU gagal melakukan *booting*.

- **Latihan 2 (Implementasi Kode):** Modifikasi kode pada Praktikum 1 agar LED berkedip secara *tidak simetris*, yaitu menyala sangat cepat selama 200 ms, kemudian mati agak lama selama 800 ms. Tuliskan dua baris fungsi delay() yang harus Anda ubah.

  Fungsi delay() digunakan untuk menghentikan sementara eksekusi program dalam satuan milidetik.

  Untuk menghasilkan kedipan tidak simetris (menyala 200 ms dan mati 800 ms), harus mengubah dua baris instruksi delay(1000); pada blok fungsi loop().

  Dua baris fungsi yang diubah menjadi:

  - delay(200); diletakkan setelah digitalWrite(ledPin, HIGH).
  - delay(800); diletakkan setelah digitalWrite(ledPin, LOW).

```
9   void loop() {
10    digitalWrite(ledPin, HIGH);
11    Serial.println("LED Menyala");
12    delay(200);
13
14    digitalWrite(ledPin, LOW);
15    Serial.println("LED Mati");
16    delay(800);
17  }
```

- **Latihan 3 (Analisis Sirkuit):** Berdasarkan hukum aliran arus listrik searah, apa yang terjadi jika pada Praktikum 1 Anda memasang kaki LED secara terbalik (Anoda ke GND, Katoda ke D1)? Apakah program akan mengalami *error* kompilasi? Mengapa LED tidak menyala?

  Program tidak akan mengalami *error* kompilasi karena IDE hanya mengevaluasi kebenaran sintaks (kode perangkat lunak), bukan wujud fisik atau arah pemasangan perangkat keras.

  Pada LED, kaki Anoda adalah sisi positif dan kaki Katoda adalah sisi negatif.

  Instruksi digitalWrite(ledPin, HIGH) memberikan tegangan 3.3V ke pin sebagai jalur keluar tegangan.

  Jika dipasang terbalik (sumber tegangan D1 masuk ke Katoda dan Anoda ke GND), LED tidak akan menyala karena sifat komponen dioda hanya mengizinkan arus listrik mengalir satu arah (dari positif ke negatif). Terbaliknya polaritas memblokir aliran arus sehingga sirkuit terputus.

- **Latihan 4 (Pengembangan Algoritma):** Ubah blok kondisi if-else pada Praktikum 2 sehingga perilakunya terbalik: "Saat tombol TIDAK ditekan, LED **menyala**. Saat tombol DITEKAN, LED justru **mati**."

  Pada rangkaian resistor *pull-down*, tombol yang tidak ditekan akan ditarik ke GND sehingga statusnya pasti terbaca LOW (0), sedangkan menekan tombol menghasilkan status HIGH (1).

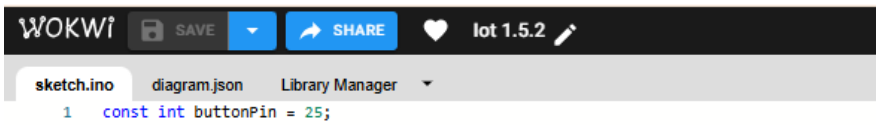
  Untuk membalik perilakunya, kondisi output di dalam logika if-else harus ditukar posisi penempatan perintah HIGH dan LOW-nya:

```
15  if (buttonState == HIGH) {
16    digitalWrite(ledPin, LOW); // LED justru mati saat tombol ditekan
17    Serial.println("Tombol ditekan! -> LED OFF");
18  } else {
19    digitalWrite(ledPin, HIGH); // LED menyala saat tombol dilepas (LOW)
20  }
```

1.7 Tugas - Modifikasi Sistem Sakelar *Toggle* (*Latching*)

Tujuan: Mampu mengimplementasikan logika pelacakan status (*state tracking*) dan membedakan antara perilaku tombol *momentary* dengan *latching*.

KODE PROGRAM




```
1   const int buttonPin = 25;
2   const int ledPin = 26;
3
4   // Variabel untuk melacak status LED saat ini (state tracking)
5   // false = LED mati (OFF), true = LED menyala (ON)
6   bool ledState = false;
7
8   // Variabel untuk menyimpan status tombol pada pembacaan sebelumnya,
9   // digunakan untuk mendeteksi transisi LOW -> HIGH (baru ditekan),
10  // bukan sekadar status tombol saat ini yang bisa saja masih ditahan.
11  int lastButtonState = LOW;
12
13  void setup() {
14    Serial.begin(115200);
```

```ino
void setup() {
  Serial.begin(115200);
  pinMode(buttonPin, INPUT);
  pinMode(ledPin, OUTPUT);

  // Pastikan LED mati saat sistem pertama kali dinyalakan
  digitalWrite(ledPin, ledState);
}

void loop() {
  // Baca status tombol saat ini
  int currentButtonState = digitalRead(buttonPin);

  // --- LOGIKA STATE TRACKING (TOGGLE) ---
  // Perubahan status hanya diproses ketika terjadi transisi dari
  // LOW (tidak ditekan) ke HIGH (baru ditekan). Dengan cara ini,
  // menahan tombol tetap ditekan tidak akan membuat LED
  // berubah status berulang kali (bukan momentary).
  if (currentButtonState == HIGH && lastButtonState == LOW) {

    // --- LOGIKA DEBOUNCE ---
    // Saat tombol fisik ditekan, kontak logamnya bisa "memantul"
    // beberapa kali dalam hitungan milidetik, menyebabkan
    // digitalRead() membaca beberapa transisi palsu.
    // delay(200) di sini memberi jeda singkat agar sinyal tombol
    // benar-benar stabil sebelum status LED diproses lebih lanjut,
    // sehingga mencegah flickering / toggle ganda yang tidak diinginkan.
    delay(200);

    // Balik (toggle) status LED: ON jadi OFF, OFF jadi ON
    ledState = !ledState;
    digitalWrite(ledPin, ledState);

    // Balik (toggle) status LED: ON jadi OFF, OFF jadi ON
    ledState = !ledState;
    digitalWrite(ledPin, ledState);

    if (ledState) {
      Serial.println("Tombol ditekan! -> LED ON");
    } else {
      Serial.println("Tombol ditekan! -> LED OFF");
    }
  }

  // Simpan status tombol saat ini sebagai status sebelumnya
  // untuk perbandingan pada iterasi loop() berikutnya
  lastButtonState = currentButtonState;
}
```

CARA KERJA PROGRAM

- Berbeda dengan versi *momentary* pada kode sebelumnya di mana kondisi tegangan *output* hanya dieksekusi selaras dengan tekanan tombol saat itu juga, program ini diubah untuk berfungsi layaknya sakelar lampu ruangan.
- Saat sistem mikrokontroler pertama kali dinyalakan, LED berada dalam keadaan mati secara *default*.
- Jika tombol ditekan satu kali secara singkat lalu dilepas, mikrokontroler mendeteksi adanya transisi status, menyalakan LED, dan mempertahankannya dalam kondisi menyala (ON).

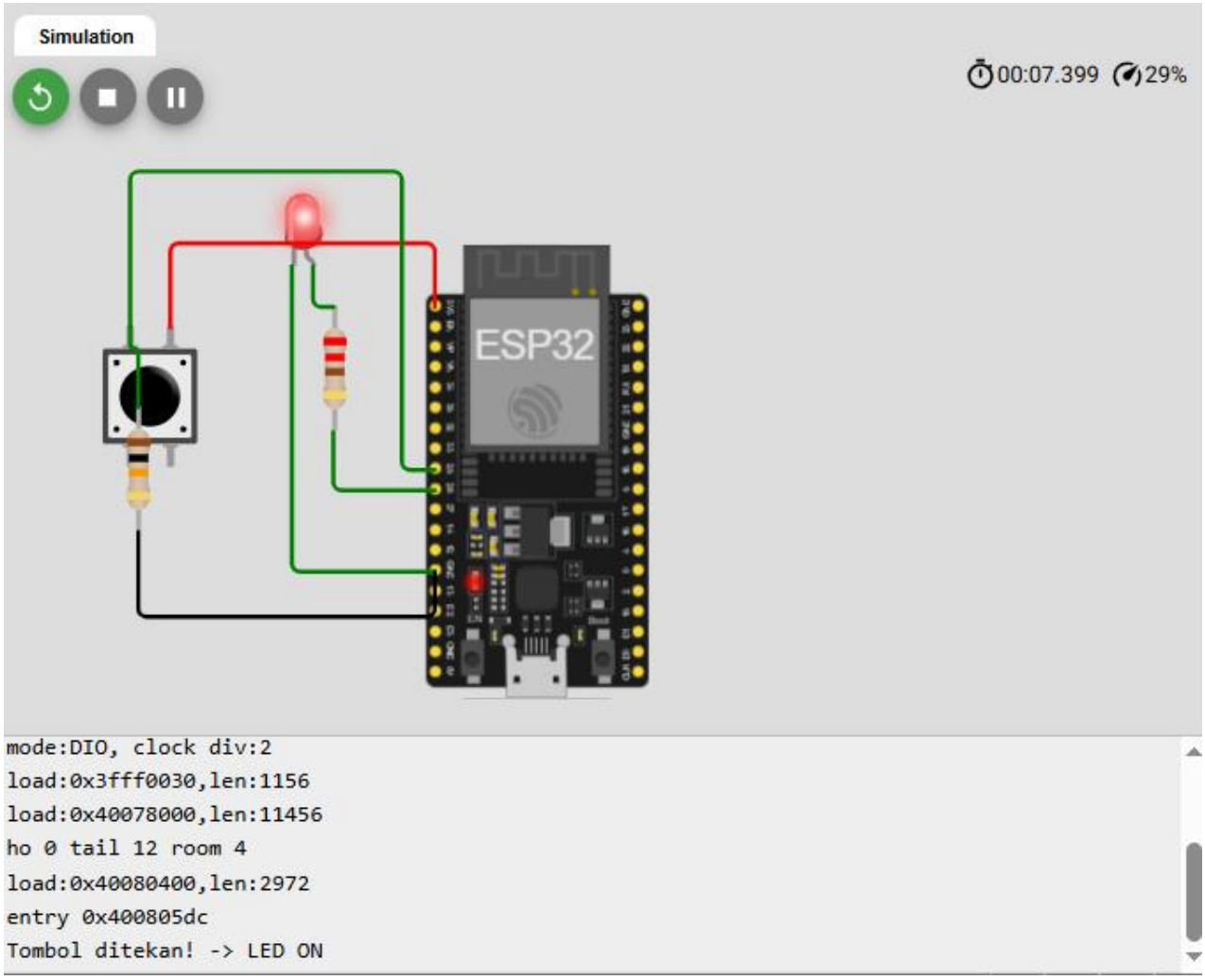
- Jika tombol kembali ditekan satu kali lalu dilepas, mikrokontroler akan merespons transisi tersebut dengan mematikan LED dan mempertahankannya dalam kondisi mati (OFF).

PENJELASAN SINGKAT KODE

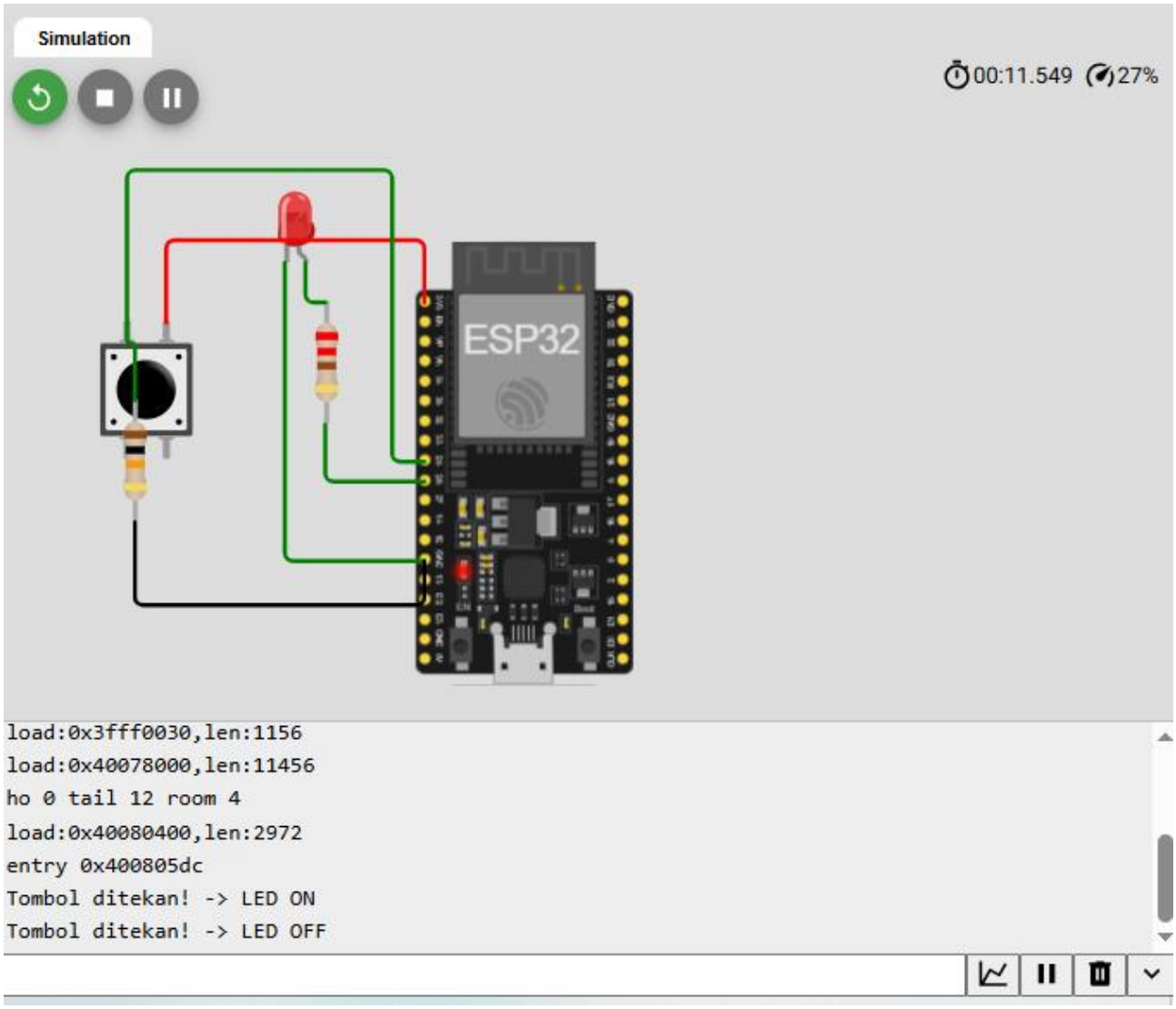
- bool ledState = false;: Deklarasi variabel penyimpanan status (*state tracking*) yang bertugas melacak secara absolut apakah lampu saat ini sedang ON atau OFF.
- int lastButtonState = LOW;: Menyimpan rekaman pembacaan tombol pada iterasi sebelumnya. Digunakan sebagai dasar perbandingan logika transisi.
- if (currentButtonState == HIGH && lastButtonState == LOW): Kondisi logika ini membedakan respons tombol. Program hanya merespons tepat di momen tombol berubah dari tidak ditekan menjadi ditekan, sehingga menahan tombol tidak akan memicu eksekusi berulang.
- delay(200);: Instruksi jeda singkat yang krusial untuk metode *debouncing*. Menahan program sesaat guna mengatasi fenomena pantulan mekanis (*bouncing*) pada komponen kontak logam fisik di dalam *push button*.
- ledState = !ledState;: Logika pembalik (inversi). Jika nilai sebelumnya true (ON), maka akan diubah menjadi false (OFF), dan sebaliknya.

GAMBAR

Saat pertama kali push button di klik sekali dan led menyala terus.



Saat push button di klik yang kedua kali dan led mati.



HASIL: Sistem berjalan secara stabil sesuai kriteria keberhasilan yang ditargetkan. Perubahan pergantian status dari ON ke OFF dan sebaliknya terjadi sempurna hanya dengan mengandalkan satu kali tekanan singkat (bukan ditahan). Penambahan logika *debouncing* juga berhasil mengeliminasi fenomena *flickering* sehingga lampu tidak pernah berkedip acak atau gagal mendeteksi input ketika ditekan.